To handle the request, you can use the directive `parameters`. Pass a list of names and types to get the `parameter` extraction:

```
object HttpServerShardedContainer {
  ...
    post {                                                                  ❶
      parameters("entityId".as[String],"kind".as[String],"size".as[Int]     ❷
             (entityId, kind, size) =>
```

❶ Method directive

❷ Parameters directive

With the extraction `(entityId, kind, size)`, you can create the `Cargo` and then send it to the container. You can `tell` and reply to the user immediately or `ask` and wait for the container to reply. The choice depends on the business case. In this scenario, let's prioritize performance and opt for `tell` rather than informing the user only after ensuring that the cargo has been added by using `ask`. This looks like the following:

```
            (entityId, kind, size) =>

            val cargo = Container.Cargo(kind, size)     ❶
            shardingRegion ! ShardingEnvelope(
              entityId,
              Container.AddCargo(cargo))                ❷
  ...
}
```

❶ Creates the cargo

❷ Adds the cargo to the container

Transforming the response